

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

Báo Cáo Nhập môn An toàn thông tin

Đề tài: Các cơ chế phát hiện xâm nhập mạng

Học phần: Nhập môn An toàn thông tin

Mã học phần: IT4015

Giảng viên hướng dẫn: PGS. TS. Nguyễn Linh Giang

Sinh viên thực hiện: Nguyễn Đình Tuấn Minh 20230048

Vũ Đức Tâm 20230064

Nguyễn Gia Khánh 202416803

Nguyễn Đăng Cao Tuấn 202400119

Hà Nội, June 24, 2026

Mục lục

Lời nói đầu	3
1 Giới thiệu đề tài	4
1.1 Bối cảnh và lý do chọn đề tài	4
1.2 Mục tiêu đề tài	4
1.3 Đối tượng và phạm vi nghiên cứu	4
1.4 Phương pháp thực hiện	5
1.5 Bố cục báo cáo	5
2 Cơ sở lý thuyết	6
2.1 IDS, IPS và NIDS	6
2.2 Phát hiện theo chữ ký và Chỉ số thỏa hiệp IOC	7
2.3 Thuật toán so khớp chuỗi trong NIDS	8
2.3.1 Thuật toán Boyer-Moore	8
2.3.2 Thuật toán Aho-Corasick	8
2.3.3 Vấn đề quay lui của Biểu thức chính quy và ReDoS	8
2.4 Phát hiện theo hành vi và Cửa sổ trượt	9
2.5 Cơ chế chống lụt cảnh báo bằng thời gian hạ nhiệt	9
2.6 Phân tích cấu trúc và tách lọc giao thức HTTP	10
2.6.1 Cơ chế đối sánh Sticky Buffer và an toàn ứng dụng Web	10
2.7 Các kiểu tấn công được mô hình hóa trong bài tập lớn	10
2.7.1 Quét cổng	10
2.7.2 Tấn công từ chối dịch vụ ICMP Flood	11
2.7.3 Tấn công từ chối dịch vụ TCP SYN Flood	11
2.7.4 Kênh truyền độc hại qua DNS	11
2.8 Giới hạn lý thuyết của việc triển khai NIDS	12
3 Kiến trúc và Triển khai Hệ thống NIDS	13
3.1 Tổng quan kiến trúc	13
3.2 Thu thập và chuẩn hóa gói tin	13
3.2.1 Capture	13
3.2.2 PacketEvent	14
3.3 Bộ giải mã giao thức và chuẩn hóa dữ liệu	14
3.4 Detection Engine	15
3.5 Luật phát hiện tích hợp	15
3.5.1 Cơ chế chống tấn công State Exhaustion DoS	15
3.6 Luật chữ ký dạng Suricata-style subset	16
3.7 Lưu trữ và Dashboard hiển thị	17
3.7.1 Lưu trữ cảnh báo dạng JSON Lines	17
3.7.2 Cơ chế hiển thị trên Dashboard Flask	17

4	Kịch bản Thử nghiệm và Đánh giá	18
4.1	Môi trường thử nghiệm	18
4.2	Khởi động hệ thống	18
4.3	Các kịch bản kiểm thử	19
4.3.1	Quét cổng bằng nmap	19
4.3.2	ICMP Ping Flood	19
4.3.3	TCP SYN Flood	19
4.3.4	DNS tunneling	19
4.3.5	DNS signature rule	20
4.4	Phân tích cấu trúc nhật ký cảnh báo sinh ra (JSONL)	20
4.4.1	Cảnh báo hành vi: Quét cổng (Port Scan)	20
4.4.2	Cảnh báo hành vi: ICMP Ping Flood	20
4.4.3	Cảnh báo hành vi: TCP SYN Flood	21
4.4.4	Cảnh báo bất thường: DNS Tunneling	21
4.5	Đánh giá kết quả	22
5	Kết luận và Hướng phát triển	23
5.1	Kết quả đạt được	23
5.2	Hạn chế	23
5.3	Hướng phát triển	24
5.4	Kết luận	24
	Phụ lục	25
	Tài liệu tham khảo	28

Lời nói đầu

Trong kỷ nguyên số hóa, hệ thống mạng là nền tảng cho hầu hết hoạt động của tổ chức và doanh nghiệp. Cùng với lợi ích đó, các rủi ro an ninh mạng cũng xuất hiện thường xuyên hơn, đặc biệt là các hành vi thăm dò, khai thác dịch vụ và truyền dữ liệu bất thường qua những giao thức hợp lệ.

Trong bối cảnh đó, các biện pháp phòng thủ tĩnh như tường lửa vẫn cần thiết nhưng chưa đủ để quan sát nội dung và hành vi của lưu lượng mạng. Hệ thống phát hiện xâm nhập mạng NIDS giúp bổ sung lớp giám sát, tạo cảnh báo khi lưu lượng có dấu hiệu bất thường hoặc khớp với mẫu tấn công đã biết.

Trong khuôn khổ học phần **IT4015 - Nhập môn An toàn thông tin**, nhóm em lựa chọn thực hiện đề tài: **"Các cơ chế phát hiện xâm nhập mạng"**. Trọng tâm của bài tập lớn là xây dựng một NIDS nhỏ gọn bằng Python để minh họa quy trình thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký và hiển thị cảnh báo.

Mục tiêu của báo cáo là trình bày ngắn gọn cơ sở lý thuyết cần thiết, mô tả kiến trúc hệ thống đã triển khai, kiểm thử trong môi trường lab và chỉ ra các giới hạn còn tồn tại.

Mặc dù đã nỗ lực tìm hiểu và tổng hợp tài liệu, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, bản báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm em rất mong nhận được sự nhận xét và góp ý quý báu từ thầy để kiến thức của nhóm chúng em được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 24 tháng 06 năm 2026

Nhóm sinh viên thực hiện

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và lý do chọn đề tài

Trong các hệ thống mạng hiện đại, tường lửa chỉ là một lớp bảo vệ ban đầu. Nhiều cuộc tấn công vẫn đi qua các cổng hợp lệ như HTTP, DNS hoặc HTTPS, sau đó khai thác điểm yếu ở tầng ứng dụng hoặc tạo ra các hành vi bất thường trong mạng nội bộ. Vì vậy, hệ thống phát hiện xâm nhập mạng NIDS là một thành phần quan trọng để giám sát lưu lượng, phát hiện dấu hiệu tấn công và hỗ trợ quản trị viên phản ứng kịp thời.

Đề tài này tập trung xây dựng một hệ thống NIDS nhỏ gọn bằng ngôn ngữ Python nhằm minh họa các thành phần cốt lõi của một hệ thống phát hiện xâm nhập: thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký, lưu trữ cảnh báo và hiển thị qua giao diện Dashboard. Mục tiêu chính là phục vụ học tập và trình diễn kỹ thuật, không hướng tới thay thế các hệ thống thương mại lớn như Suricata hoặc Zeek.

1.2 Mục tiêu đề tài

Bài tập đặt ra các mục tiêu sau:

- **Thu thập và chuẩn hóa gói tin:** Đọc lưu lượng từ tệp PCAP hoặc giao diện mạng trực tiếp bằng thư viện Scapy, sau đó chuyển đổi thành cấu trúc sự kiện thống nhất.
- **Phát hiện theo hành vi:** Triển khai các luật dựa trên cửa sổ thời gian để nhận diện quét cổng, ICMP flood, TCP SYN flood và dấu hiệu DNS tunneling.
- **Phát hiện theo chữ ký và Chỉ số thỏa hiệp IOC:** Xây dựng bộ phân tích cú pháp cho một tập con cú pháp kiểu Suricata và kiểm tra các mẫu chỉ số thỏa hiệp IOC như danh sách đen tên miền hoặc truy vấn DNS đã biết.
- **Ghi nhận và trực quan hóa cảnh báo:** Lưu cảnh báo dưới dạng JSON Lines và hiển thị thống kê qua giao diện web Flask.
- **Kiểm thử trong môi trường thực nghiệm:** Mô phỏng các luồng tấn công phổ biến bằng công cụ tấn công trên hệ điều hành Kali Linux để đánh giá khả năng kích hoạt cảnh báo của hệ thống.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là hệ thống phát hiện xâm nhập mạng dạng thụ động. Hệ thống quan sát lưu lượng, tạo cảnh báo và phục vụ phân tích, nhưng không trực tiếp chặn gói tin hoặc ngắt phiên kết nối như một hệ thống ngăn chặn xâm nhập IPS.

Phạm vi triển khai của bài tập lớn được giới hạn như sau:

- Hệ thống xử lý các trường thông tin phổ biến của giao thức IP, TCP, UDP, ICMP và DNS.
- Phần phát hiện hành vi sử dụng ngưỡng tĩnh và cửa sổ trượt thời gian; các ngưỡng này là các giá trị lab/demo, cần hiệu chỉnh theo đặc tính của từng mạng thực tế.
- Phần chữ ký hỗ trợ một Suricata-style subset, đủ cho tình huống demo khớp tên miền chỉ số thỏa hiệp IOC mẫu trong truy vấn DNS.
- Hệ thống chưa giải mã TLS hoặc HTTPS, chưa tái tạo đầy đủ luồng dữ liệu TCP stream và chưa được tối ưu cho lưu lượng mạng tốc độ cao.

1.4 Phương pháp thực hiện

Đề tài được triển khai theo hướng kết hợp lý thuyết và thực nghiệm:

- Nghiên cứu các cơ chế IDS cơ bản, đặc biệt là phát hiện theo chữ ký và phát hiện theo hành vi.
- Thiết kế kiến trúc dạng đường ống dẫn dữ liệu gồm các khối thu thập gói tin, bộ giải mã và phân tích cú pháp, động cơ luật phát hiện, bộ lưu trữ và giao diện hiển thị.
- Cài đặt từng module bằng Python, sử dụng Scapy cho xử lý gói tin và Flask cho giao diện web.
- Viết luật kiểm thử và kịch bản tấn công mô phỏng để kiểm tra kết quả cảnh báo.
- Đối chiếu kết quả với phạm vi đã đặt ra, đồng thời chỉ ra hạn chế còn tồn tại.

1.5 Bố cục báo cáo

Báo cáo gồm 5 chương:

- **Chương 1: Giới thiệu đề tài** – trình bày bối cảnh, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương 2: Cơ sở lý thuyết** – tóm tắt các khái niệm IDS, NIDS, thuật toán so khớp chữ ký và phát hiện theo hành vi.
- **Chương 3: Kiến trúc và triển khai hệ thống** – mô tả các module chính, cơ chế bộ giải mã chuẩn hóa dữ liệu và cách dữ liệu đi qua hệ thống.
- **Chương 4: Kịch bản thử nghiệm và đánh giá** – trình bày môi trường phòng thí nghiệm, các kịch bản tấn công và phân tích cấu trúc nhật ký cảnh báo chi tiết.
- **Chương 5: Kết luận và hướng phát triển** – tổng kết kết quả đạt được, hạn chế và hướng cải tiến.

Chương 2

Cơ sở lý thuyết

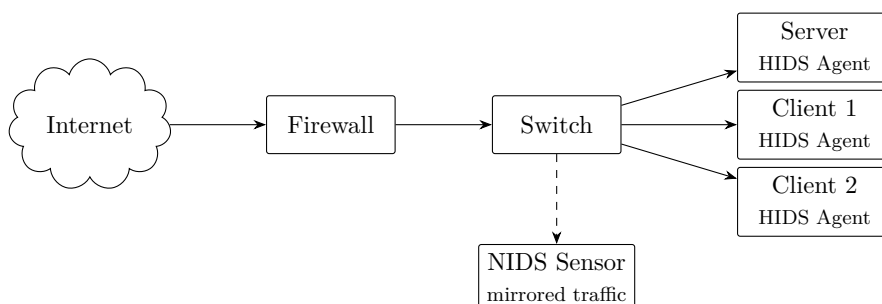
2.1 IDS, IPS và NIDS

Hệ thống phát hiện xâm nhập IDS là một giải pháp giám sát an ninh mạng có nhiệm vụ thu thập, phân tích thông tin từ các nguồn khác nhau trong hệ thống công nghệ thông tin để phát hiện hành vi vi phạm chính sách bảo mật hoặc các dấu hiệu tấn công. IDS hoạt động theo cơ chế thụ động: quan sát dữ liệu đi qua, ghi nhật ký và phát cảnh báo đến quản trị viên mà không trực tiếp thay đổi hoặc can thiệp vào luồng gói tin thực tế.

Ngược lại với IDS, Hệ thống ngăn chặn xâm nhập IPS hoạt động ở chế độ chủ động. IPS thường được triển khai nằm trực tiếp trên đường truyền. Khi phát hiện gói tin độc hại hoặc lưu lượng bất thường, IPS có khả năng can thiệp trực tiếp bằng cách hủy gói tin, ngắt kết nối hoặc cấu hình lại tường lửa để chặn địa chỉ IP nguồn của kẻ tấn công ngay lập tức. Trong phạm vi bài tập lớn này, nhóm tập trung nghiên cứu và triển khai theo hướng IDS để tối ưu hóa hiệu năng giám sát thụ động.

Dựa trên nguồn dữ liệu và vị trí giám sát, IDS được chia thành hai trường phái chính:

- **NIDS:** Giám sát toàn bộ lưu lượng mạng đi qua một hoặc nhiều phân đoạn mạng. NIDS thu thập dữ liệu bằng cách lắng nghe trực tiếp thông qua card mạng đặt ở chế độ promiscuous mode, hoặc qua cổng cấu hình Span/Mirror port trên Switch, hoặc thiết bị TAP vật lý. Ưu điểm của NIDS là tính độc lập cao, không làm ảnh hưởng đến hiệu năng của các máy chủ dịch vụ, và có khả năng giám sát đồng thời nhiều thiết bị trong mạng.
- **HIDS:** Cài đặt trực tiếp dưới dạng các agent trên từng hệ điều hành máy chủ hoặc máy trạm endpoint. HIDS thu thập dữ liệu từ log hệ thống, log ứng dụng, tính toàn vẹn của các file cấu hình quan trọng thông qua FIM, các cuộc gọi hệ thống system calls và các tiến trình đang chạy. HIDS có ưu điểm lớn trong việc phát hiện các cuộc tấn công leo thang đặc quyền nội bộ hoặc các gói tin độc hại đã được giải mã ngay tại máy chủ, nhưng đòi hỏi tài nguyên cài đặt và quản lý phức tạp trên diện rộng.



Hình 2.1: Minh họa vị trí giám sát của NIDS và HIDS

Loại hệ thống	Nguồn dữ liệu	Đặc điểm
NIDS	Lưu lượng mạng tại switch, gateway hoặc interface giám sát.	Quan sát được nhiều máy trong cùng phân đoạn mạng, phù hợp phát hiện scan, flood và payload độc hại trong lưu lượng không mã hóa.
HIDS	Log, tiến trình, file và hành vi trên từng máy chủ hoặc endpoint.	Nhìn sâu vào trạng thái máy trạm nhưng cần cài agent trên từng host.

Bảng 2.1: Phân loại IDS theo nguồn dữ liệu giám sát

Hệ thống trong bài tập là một NIDS dạng thụ động: đọc PCAP hoặc sniff interface, phân tích gói tin và ghi cảnh báo ra tệp JSONL.

2.2 Phát hiện theo chữ ký và Chỉ số thỏa hiệp IOC

Phát hiện theo chữ ký là phương pháp đối sánh dữ liệu lưu lượng mạng với một tập hợp các mẫu hoặc luật đã được định nghĩa trước về hành vi độc hại. Các chữ ký này thường được trích xuất từ các Chỉ số thỏa hiệp IOC bao gồm: chuỗi nhị phân đặc trưng của mã độc, địa chỉ IP thuộc mạng botnet, tên miền độc hại của máy chủ điều khiển C2, hoặc các mẫu chuỗi ký tự trong giao thức ứng dụng.

Ưu điểm lớn nhất của phương pháp này là tốc độ xử lý nhanh đối với các cuộc tấn công đã biết, tỷ lệ false positive cực thấp và cung cấp thông tin chi tiết về loại tấn công giúp kiểm chứng dễ dàng. Tuy nhiên, điểm hạn chế chí mạng là hoàn toàn không thể phát hiện các cuộc tấn công Zero-day hoặc các biến thể mã độc đã được thực hiện obfuscation hoặc mã hóa lưu lượng.

Trong dự án này, nhóm đã tự triển khai một công cụ phân tích cú pháp và engine nạp luật tương thích với định dạng luật của Suricata/Snort. Một luật chuẩn bao gồm hai phần:

- **Header:** Khai báo hành động (thường là `alert`), giao thức mạng (như `tcp`, `udp`, `icmp`, `dns`, `http`), dải địa chỉ IP nguồn/đích, cổng dịch vụ nguồn/đích và hướng của luồng dữ liệu (`->` một chiều, hoặc `<>` hai chiều).
- **Rule Options:** Được bao quanh bởi dấu ngoặc đơn, chứa các từ khóa điều khiển:
 - `msg`: Chuỗi mô tả thông tin cảnh báo xuất ra log.
 - `content`: Chuỗi ký tự nhị phân hoặc văn bản cần tìm kiếm trong gói tin.
 - `pcree`: Biểu thức chính quy giúp khớp các mẫu phức tạp hơn.
 - `dns.query`: Từ khóa chỉ định vị trí đối sánh thuộc phần truy vấn của giao thức DNS.
 - `http.uri`, `http.method`, `http.header`: Các sticky buffer chứa dữ liệu HTTP để tăng hiệu năng đối sánh.
 - `sid`: Định danh duy nhất cho luật đó.

Ví dụ một luật mẫu phát hiện truy vấn tên miền phục vụ kiểm thử độc hại:

```
alert dns any any -> any 53 (msg:"Custom DNS Query: example.com";
dns.query; content:"example.com"; nocase; sid:1000101; rev:1;)
```


2.3 Thuật toán so khớp chuỗi trong NIDS

Trong các hệ thống NIDS dựa trên chữ ký, nhiệm vụ quan trọng nhất và tiêu tốn nhiều tài nguyên CPU nhất là tìm kiếm hàng ngàn chuỗi ký tự độc hại trong phần payload của hàng triệu gói tin đi qua mỗi giây. Để giải quyết bài toán này, các hệ thống IDS/IPS hiện đại như Suricata hay Snort không sử dụng phương pháp duyệt tuần tự thông thường mà áp dụng các thuật toán so khớp chuỗi tối ưu hóa cao:

2.3.1 Thuật toán Boyer-Moore

Thuật toán Boyer-Moore là thuật toán tìm kiếm chuỗi đơn hiệu quả nhất trong thực tế. Khác với thuật toán tìm kiếm thông thường, Boyer-Moore thực hiện so khớp các ký tự từ phải sang trái của mẫu nhưng dịch chuyển mẫu từ trái sang phải trên chuỗi văn bản cần tìm kiếm. Thuật toán tận dụng thông tin thu được từ các ký tự không khớp thông qua hai bảng dịch chuyển:

- **Bad Character Rule:** Khi xảy ra lỗi không khớp tại ký tự $T[i] \neq P[j]$, thuật toán sẽ tìm kiếm ký tự $T[i]$ trong mẫu P . Nếu có, dịch chuyển mẫu để ký tự $T[i]$ trong mẫu trùng khớp với $T[i]$ trong văn bản. Nếu không có, dịch chuyển toàn bộ mẫu vượt qua vị trí $T[i]$.
- **Good Suffix Rule:** Nếu một phần hậu tố của mẫu khớp với văn bản trước khi xảy ra lỗi không khớp, thuật toán sẽ dịch chuyển mẫu đến vị trí tiếp theo mà hậu tố này xuất hiện lại trong mẫu.

Nhờ hai quy tắc này, Boyer-Moore cho phép bỏ qua rất nhiều ký tự trong văn bản mà không cần kiểm tra, đạt độ phức tạp thời gian tốt nhất là $O(n/m)$ với n là độ dài văn bản và m là độ dài mẫu.

2.3.2 Thuật toán Aho-Corasick

Khi số lượng chữ ký tăng lên hàng chục ngàn, việc chạy thuật toán đơn mẫu như Boyer-Moore cho từng chữ ký trên mỗi gói tin sẽ gây ra suy giảm hiệu năng nghiêm trọng. Thuật toán Aho-Corasick giải quyết vấn đề này bằng cách xây dựng một máy trạng thái hữu hạn chứa tất cả các chữ ký cần tìm kiếm. Máy trạng thái Aho-Corasick hoạt động dựa trên ba hàm cốt lõi:

- **Goto function:** Dẫn dắt quá trình di chuyển từ trạng thái này sang trạng thái khác dựa trên ký tự tiếp theo trong văn bản. Nó tạo ra một cấu trúc cây phân cấp cho tất cả các mẫu.
- **Failure function:** Khi một ký tự không khớp với bất kỳ nhánh con nào tại trạng thái hiện tại, hàm thất bại sẽ chỉ ra trạng thái tiếp theo cần chuyển đến. Hàm này giúp thuật toán không cần phải quay lui trên chuỗi văn bản đầu vào.
- **Output function:** Liên kết các trạng thái với danh sách các mẫu được phát hiện tương ứng khi trạng thái đó được tiếp cận.

Nhờ cơ chế này, thuật toán Aho-Corasick có thể tìm kiếm đồng thời tất cả các chữ ký trong văn bản chỉ với một lần duyệt duy nhất. Độ phức tạp thời gian đạt mức tuyến tính $O(n + z)$ trong đó n là độ dài văn bản và z là số lần xuất hiện của các mẫu độc hại, hoàn toàn độc lập với số lượng chữ ký cài đặt. Đây là nền tảng của cơ chế fast_pattern trong Suricata.

2.3.3 Vấn đề quay lui của Biểu thức chính quy và ReDoS

Mặc dù so khớp chuỗi tĩnh rất nhanh, kẻ tấn công có thể dễ dàng thay đổi một vài ký tự trong payload để né tránh. Do đó, các IDS cần dùng biểu thức chính quy để mô tả các biến thể tấn công linh hoạt hơn. Tuy nhiên, việc sử dụng các công cụ regex truyền thống dựa trên NFA có

khả năng gây ra hiện tượng quay lui thảm khốc. Khi gộp một chuỗi đầu vào không khớp được thiết kế đặc biệt, động cơ regex sẽ thử tất cả các tổ hợp quay lui có thể, dẫn đến độ phức tạp thời gian tăng theo cấp số nhân $O(2^n)$. Kẻ tấn công có thể lợi dụng lỗ hổng này để thực hiện tấn công từ chối dịch vụ vào chính hệ thống IDS. Vì lý do này, các IDS hiện đại thường sử dụng công cụ regex của Google hoạt động dựa trên DFA để đảm bảo thời gian chạy tuyến tính, hoặc hạn chế nghiêm ngặt việc sử dụng các mẫu regex có cấu trúc lặp lồng nhau trong các file luật.

2.4 Phát hiện theo hành vi và Cửa sổ trượt

Khác với phát hiện theo chữ ký tập trung vào một gói tin đơn lẻ, Phát hiện theo hành vi tập trung phân tích xu hướng hoặc mối liên hệ giữa các gói tin theo chuỗi thời gian. Phương pháp này đặc biệt hiệu quả trong việc nhận diện các kiểu tấn công do thám hoặc từ chối dịch vụ vốn không mang nội dung độc hại trong từng gói tin riêng lẻ nhưng lại tạo ra lưu lượng bất thường khi gộp lại.

Để thực hiện điều này một cách tối ưu trong môi trường thời gian thực mà không làm tràn bộ nhớ, kỹ thuật cửa sổ trượt thời gian được áp dụng.

- Hệ thống duy trì một hàng đợi hai đầu lưu trữ dấu vết thời gian của các gói tin nhận được từ mỗi địa chỉ IP nguồn trong khoảng thời gian T giây.
- Với mỗi gói tin mới đến từ một nguồn, hệ thống tiến hành loại bỏ các bản ghi cũ có $t < t_{\text{current}} - T$ ở đầu hàng đợi.
- Sau đó, thêm gói tin hiện tại vào cuối hàng đợi và kiểm tra xem tổng số gói tin hoặc số lượng thuộc tính duy nhất có vượt quá ngưỡng N quy định hay không. Nếu có, phát cảnh báo về hành vi bất thường.

Phương pháp này giúp phát hiện được các hành vi nguy hại mà không cần biết trước signature của cuộc tấn công, tuy nhiên thách thức lớn nhất là xác định ngưỡng N và T tối ưu cho từng hạ tầng mạng thực tế. Nếu đặt ngưỡng quá nhạy sẽ dẫn đến báo động giả hàng loạt; ngược lại, nếu đặt ngưỡng quá cao sẽ làm mất tác dụng cảnh báo của hệ thống.

2.5 Cơ chế chống lụt cảnh báo bằng thời gian hạ nhiệt

Trong các đợt tấn công từ chối dịch vụ tốc độ cao hoặc quét cổng quy mô lớn, kẻ tấn công có thể gửi hàng vạn gói tin mỗi giây. Nếu hệ thống NIDS phát sinh cảnh báo cho từng gói tin riêng lẻ, điều này sẽ dẫn đến những hậu quả nghiêm trọng:

- **Tràn ngập cảnh báo:** Hệ thống ghi nhận quá nhiều bản ghi giống nhau trong thời gian ngắn, làm cạn kiệt dung lượng đĩa lưu trữ và gây quá tải hệ thống hiển thị Dashboard.
- **Suy giảm hiệu năng I/O:** Thao tác ghi đĩa liên tục làm chậm toàn bộ tiến trình phân tích gói tin, gây rơi rớt gói tin trên card mạng.
- **Mất tập trung cho quản trị viên:** Quản trị viên an ninh mạng bị quá tải thông tin và khó nhận diện các mối đe dọa khác.

Để giải quyết vấn đề này, hệ thống triển khai cơ chế hạ nhiệt cảnh báo thông qua bảng tra cứu thời gian gần nhất của mỗi cặp luật và khóa nguồn. Khi một cảnh báo được kích hoạt, mốc thời gian của nó sẽ được ghi lại. Trong vòng 5 giây tiếp theo, tất cả các cảnh báo trùng lặp từ cùng một nguồn IP cho cùng một luật sẽ bị triệt tiêu hoàn toàn. Cơ chế này giúp gom các sự kiện lặp lại thành một cảnh báo duy nhất mà vẫn đảm bảo tính chính xác về số liệu thống kê gói tin.

2.6 Phân tích cấu trúc và tách lọc giao thức HTTP

Giao thức HTTP là mục tiêu phổ biến của các cuộc tấn công web như SQL Injection hoặc Cross-Site Scripting. Để phân tích hiệu quả lưu lượng HTTP mà không cần tải toàn bộ gói tin thô vào bộ nhớ, hệ thống triển khai bộ phân tích cú pháp HTTP nhỏ gọn hoạt động tại tầng ứng dụng:

- Bộ phân tích tiến hành cắt dòng payload thô bằng ký tự xuống dòng CRLF đặc trưng của HTTP.
- Dòng đầu tiên được tách để xác định phương thức HTTP và URI yêu cầu.
- Các dòng tiếp theo được phân tích theo định dạng khóa-giá trị phân tách bởi dấu hai chấm để trích xuất các trường thông tin quan trọng như Host và User-Agent.
- Các trường dữ liệu này sau đó được ánh xạ vào các vùng đệm chuyên biệt trong lớp quản lý bộ đệm sự kiện. Việc tách biệt các vùng đệm này giúp các luật Suricata chỉ thực hiện so khớp trên đúng phân vùng thông tin được yêu cầu, giảm thiểu tối đa các cảnh báo giả do trùng khớp ký tự ngẫu nhiên trong các phần khác của gói tin.

2.6.1 Cơ chế đối sánh Sticky Buffer và an toàn ứng dụng Web

Trong NIDS cổ điển, việc đối sánh chữ ký được thực hiện trên toàn bộ payload thô của gói tin. Tuy nhiên, điều này tạo ra một lỗ hổng nghiêm trọng gọi là evasion by misplacement. Kẻ tấn công có thể chèn các chuỗi ký tự độc hại như UNION SELECT vào trong một tiêu đề HTTP không thực thi như Cookie hoặc User-Agent để đánh lừa hệ thống. Máy chủ web khi nhận gói tin sẽ bỏ qua chuỗi này ở tiêu đề và chỉ xử lý URI. Kết quả là NIDS báo động giả hoặc bỏ lọt hành vi tấn công thực sự.

Để khắc phục điều này, NIDS hiện đại thiết lập khái niệm sticky buffer. Khi bộ phân tích cú pháp HTTP hoàn tất việc phân tách gói tin, các dữ liệu được đẩy vào các vùng nhớ đệm riêng biệt gồm:

- **http.method:** Chỉ chứa phương thức yêu cầu như GET, POST, PUT.
- **http.uri:** Chỉ chứa đường dẫn truy cập tài nguyên.
- **http.header:** Chứa toàn bộ phần tiêu đề HTTP.
- **http.user_agent:** Chứa định danh trình duyệt gửi yêu cầu.

Khi một luật Suricata chỉ định từ khóa **http.uri**, engine đối sánh sẽ chỉ thực hiện tìm kiếm chuỗi trong vùng đệm **http.uri** chứ không tìm trên toàn bộ gói tin. Cơ chế này loại bỏ hoàn toàn các trường hợp khớp chuỗi nhầm lẫn ở các tiêu đề không liên quan, nâng cao độ chính xác phát hiện lên nhiều lần.

2.7 Các kiểu tấn công được mô hình hóa trong bài tập lớn

Bài tập lớn thực hiện nghiên cứu, mô phỏng và phát hiện 4 dạng tấn công mạng phổ biến dưới góc độ lý thuyết giao thức:

2.7.1 Quét cổng

Quét cổng là giai đoạn do thám đầu tiên trong vòng đời tấn công. Kẻ tấn công gửi các gói tin thăm dò tới một dải cổng dịch vụ của mục tiêu nhằm xác định trạng thái cổng và hệ điều hành của nạn nhân. Các kỹ thuật quét cổng phổ biến bao gồm:

- **TCP SYN Scan:** Kẻ tấn công gửi gói tin với cờ SYN thiết lập. Nếu nhận được SYN-ACK từ mục tiêu, cổng đó đang mở; kẻ tấn công lập tức gửi gói RST để đóng kết nối thay vì hoàn thành cái bắt tay ba bước. Điều này giúp quét nhanh và tránh bị ghi log bởi các ứng dụng ở tầng trên.
- **TCP FIN Scan:** Gửi gói tin chỉ có cờ FIN. Theo đặc tả giao thức, nếu cổng đóng, hệ điều hành sẽ phản hồi bằng gói RST; nếu cổng mở, hệ điều hành sẽ lờ gói tin đi.
- **TCP NULL Scan và Xmas Scan:** NULL Scan gửi gói tin không bật bất kỳ cờ TCP nào. Xmas Scan bật đồng thời các cờ FIN, PSH, và URG. Cả hai đều lợi dụng phản hồi RST của cổng đóng để xác định trạng thái cổng, vượt qua các bộ lọc tường lửa thô sơ chỉ kiểm tra cờ SYN.

Hệ thống phát hiện quét cổng bằng cách theo dõi số lượng cổng đích duy nhất mà một địa chỉ IP nguồn gửi yêu cầu đến trong vòng T giây với các tổ hợp cờ TCP tương ứng.

2.7.2 Tấn công từ chối dịch vụ ICMP Flood

ICMP Flood là hình thức tấn công từ chối dịch vụ cổ điển. Kẻ tấn công gửi liên tục các gói tin truy vấn ICMP Echo Request tới địa chỉ IP của nạn nhân với tốc độ cực cao nhằm làm nghẽn băng thông đường truyền hoặc khiến CPU của thiết bị mạng bị quá tải do phải xử lý và phản hồi liên tục bằng gói tin ICMP Echo Reply. Hệ thống phát hiện bằng cách đếm số lượng gói tin ICMP truy vấn từ một địa chỉ nguồn trong cửa sổ trượt. Việc loại trừ các gói tin ICMP Reply ra khỏi bộ đếm là cực kỳ quan trọng để tránh tình trạng báo động giả lên chính nạn nhân khi họ đang gửi phản hồi lại kẻ tấn công.

2.7.3 Tấn công từ chối dịch vụ TCP SYN Flood

TCP SYN Flood lợi dụng cơ chế hàng đợi kết nối nửa mở backlog queue trong quá trình bắt tay ba bước của giao thức TCP. Mô hình lý thuyết của cuộc tấn công diễn ra qua bốn bước sau:

1. Kẻ tấn công gửi liên tục các gói tin SYN yêu cầu kết nối với địa chỉ IP nguồn giả mạo hoặc không tồn tại.
2. Máy chủ nạn nhân tiếp nhận, đưa kết nối vào hàng đợi kết nối nửa mở backlog queue, chuyển trạng thái kết nối sang SYN-RECEIVED và phản hồi bằng gói SYN-ACK tới địa chỉ IP nguồn giả mạo đó.
3. Do địa chỉ IP nguồn là giả mạo, máy chủ sẽ không bao giờ nhận được gói tin phản hồi ACK cuối cùng từ kẻ tấn công để hoàn tất kết nối. Kết nối nửa mở này sẽ được lưu giữ trong hàng đợi cho đến khi hết thời gian chờ timeout.
4. Khi kẻ tấn công gửi hàng vạn gói tin SYN liên tục, hàng đợi kết nối nửa mở của máy chủ sẽ nhanh chóng bị lấp đầy, khiến máy chủ từ chối mọi yêu cầu kết nối TCP hợp lệ khác từ người dùng thông thường.

NIDS phát hiện cuộc tấn công này bằng cách theo dõi tần suất các gói tin chỉ bật cờ SYN mà không bật cờ ACK nhằm tới một dịch vụ cụ thể trên máy chủ mục tiêu.

2.7.4 Kênh truyền độc hại qua DNS

DNS Tunneling là kỹ thuật đóng gói lưu lượng của giao thức khác vào trong các gói tin DNS hợp lệ nhằm vượt qua các thiết bị kiểm soát biên. Vì DNS là dịch vụ cốt lõi bắt buộc phải mở trên tường lửa để phân giải tên miền, kẻ tấn công có thể lợi dụng điều này để rò rỉ dữ liệu hoặc duy trì kênh kết nối ẩn. Cơ chế hoạt động dựa trên cấu trúc phân cấp DNS: kẻ tấn công đăng

ký một tên miền và trở bản ghi Name Server về một máy chủ điều khiển do chúng kiểm soát. Khi mã độc gửi truy vấn phân giải tên miền phụ chứa dữ liệu mã hóa, truy vấn này sẽ được chuyển tiếp qua các DNS Resolver hợp lệ trên Internet và cuối cùng gửi tới máy chủ điều khiển của kẻ tấn công. Máy chủ điều khiển sẽ trích xuất phần tên miền phụ để đọc dữ liệu và phản hồi lại mã độc dưới dạng các bản ghi DNS TXT, CNAME, hoặc A chứa lệnh điều khiển mới. Hệ thống phát hiện DNS Tunneling thông qua các dấu hiệu bất thường của chuỗi truy vấn DNS:

- **Độ dài truy vấn `query_length`:** Tên miền phụ chứa dữ liệu đóng gói thường có độ dài rất lớn. Giao thức DNS giới hạn tổng độ dài truy vấn tối đa là 253 ký tự.
- **Độ dài nhãn lớn nhất `max_label_length`:** Mỗi nhãn phân tách bởi dấu chấm có độ dài lớn, giới hạn tối đa là 63 ký tự cho mỗi nhãn theo đặc tả RFC.
- **Độ hỗn loạn thông tin Shannon Entropy:** Các tên miền thông thường được đặt theo ngôn ngữ tự nhiên nên có tính lặp lại cao và độ hỗn loạn thấp. Ngược lại, dữ liệu mã hóa độc hại như Base32 hay Base64 có tính ngẫu nhiên rất lớn. Công thức toán học tính Entropy Shannon của một chuỗi ký tự S là:

$$H(S) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

Trong đó $P(x_i)$ là tần suất xuất hiện của ký tự thứ x_i trong chuỗi S . Một tên miền phụ có độ dài nhãn lớn và entropy vượt quá ngưỡng 3.8 liên tục sẽ bị hệ thống đưa vào diện nghi vấn DNS Tunneling.

2.8 Giới hạn lý thuyết của việc triển khai NIDS

Mặc dù NIDS là công cụ quan trọng, trên khía cạnh lý thuyết và thực tiễn triển khai vẫn tồn tại các giới hạn sau:

- **Mã hóa lưu lượng:** Khi các giao thức chuyển dịch sang HTTPS, DoT, DoH, NIDS hoàn toàn không thể đọc được nội dung của gói tin để thực hiện đối sánh chữ ký nếu không có giải pháp giải mã trung gian.
- **Thiếu khả năng lắp ráp luồng dữ liệu:** Các cuộc tấn công ứng dụng có thể được kẻ tấn công chia nhỏ payload qua nhiều phân đoạn TCP. Nếu NIDS phân tích từng gói tin độc lập mà không tái cấu trúc lại luồng dữ liệu TCP, hệ thống sẽ không phát hiện ra mã độc.
- **Kỹ thuật né tránh:** Kẻ tấn công có thể gửi các gói tin có thời gian sống khác nhau hoặc chồng chéo dữ liệu khiến NIDS nhận dạng một đăng nhưng hệ điều hành của nạn nhân lại xử lý một nẻo, dẫn đến việc bỏ lọt tấn công.
- **Hiệu năng phần cứng và ngôn ngữ:** Việc sử dụng các ngôn ngữ bậc cao như Python kết hợp thư viện Scapy rất tốt cho việc nghiên cứu thử nghiệm học thuật, tuy nhiên trong môi trường mạng doanh nghiệp thực tế với băng thông lớn, hệ thống sẽ gặp hiện tượng nghẽn cổ chai CPU và rơi gói tin hàng loạt. Các hệ thống thực tế phải sử dụng C/C++/Rust kết hợp các cơ chế bắt gói tin hiệu năng cao.

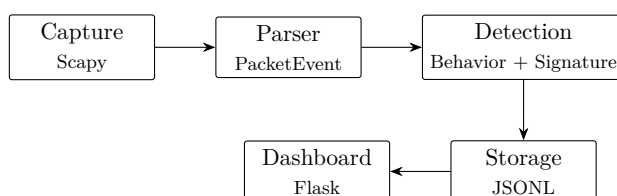
Chương 3

Kiến trúc và Triển khai Hệ thống NIDS

3.1 Tổng quan kiến trúc

Hệ thống NIDS được xây dựng bằng Python 3 và tổ chức theo mô hình pipeline. Mỗi gói tin sau khi được đọc từ PCAP hoặc interface mạng sẽ đi qua các bước: thu thập, chuẩn hóa, phát hiện, lưu trữ và hiển thị.

1. **Capture:** đọc gói tin bằng Scapy từ tệp PCAP hoặc giao diện mạng.
2. **Parser:** chuyển gói tin Scapy thành đối tượng `PacketEvent`.
3. **Detection Engine:** chạy luật hành vi/anomaly và luật chữ ký.
4. **Storage:** ghi cảnh báo ra tệp JSONL.
5. **Dashboard:** đọc dữ liệu cảnh báo và hiển thị thống kê bằng Flask.



Hình 3.1: Luồng dữ liệu của hệ thống NIDS

3.2 Thu thập và chuẩn hóa gói tin

3.2.1 Capture

Module `nids/capture.py` cung cấp hai cách lấy dữ liệu: đọc từ tệp PCAP và sniff trực tiếp trên interface. Với việc bắt gói tin trực tiếp, Scapy gọi callback cho từng gói tin; nếu parser chuyển đổi thành công thì sự kiện sẽ được đưa vào detection engine.

Đoạn mã 3.1: Live capture bằng Scapy

```
1 def sniff_live(interface: str, callback: PacketCallback, limit: int = 0) -> None:
2     from scapy.all import sniff
3
```

```

4 def handle_packet(packet: object) -> None:
5     event = packet_to_event(packet)
6     if event is not None:
7         callback(event)
8
9 sniff(iface=interface, prn=handle_packet, store=False, count=limit)

```

3.2.2 PacketEvent

Raw packet chứa nhiều trường phụ thuộc giao thức nên khó xử lý trực tiếp trong rule engine. Vì vậy, module `nids/parser.py` chuẩn hóa các trường cần thiết thành `PacketEvent` gồm thời gian, IP nguồn, IP đích, cổng nguồn, cổng đích, giao thức, các cờ TCP, loại ICMP, truy vấn DNS, nội dung văn bản gói tin và các trường HTTP phổ biến.

Đoạn mã 3.2: Cấu trúc PacketEvent

```

1 @dataclass(frozen=True)
2 class PacketEvent:
3     timestamp: float
4     src_ip: str | None
5     dst_ip: str | None
6     src_port: int | None
7     dst_port: int | None
8     protocol: str
9     length: int
10    tcp_flags: str | None = None
11    icmp_type: int | None = None
12    dns_query: str | None = None
13    payload_text: str | None = None
14    http_method: str | None = None
15    http_uri: str | None = None
16    http_host: str | None = None
17    http_user_agent: str | None = None

```

Payload được giới hạn kích thước khi chuyển thành chuỗi để tránh tốn bộ nhớ và giảm chi phí so khớp. Đây là lựa chọn phù hợp cho demo, nhưng cũng là một giới hạn khi phân tích payload lớn hoặc payload bị chia nhỏ.

3.3 Bộ giải mã giao thức và chuẩn hóa dữ liệu

Quá trình chuyển đổi từ gói tin nhị phân thô sang cấu trúc sự kiện chuẩn hóa được thực hiện tuần tự qua các bước sau:

1. Thư viện Scapy tiếp nhận luồng dữ liệu nhị phân từ card mạng hoặc tệp tin lưu trữ và ánh xạ thành các lớp đối tượng tương ứng.
2. Bộ giải mã tiến hành kiểm tra sự tồn tại của lớp giao thức IP. Tất cả các gói tin không chứa thông tin IP nguồn hoặc IP đích đều bị loại bỏ ngay lập tức để tiết kiệm tài nguyên hệ thống.
3. Tùy thuộc vào trường giao thức tại tầng vận chuyển, bộ giải mã sẽ đi sâu vào cấu trúc bên trong:
 - Đối với TCP: trích xuất cổng nguồn, cổng đích và tổ hợp các cờ trạng thái để phục vụ nhận dạng quét cổng và SYN flood.
 - Đối với UDP: trích xuất cổng nguồn và cổng đích phục vụ định danh ứng dụng.

- Đối với ICMP: trích xuất trường loại thông tin để phân loại các yêu cầu ping.
4. Tại tầng ứng dụng, nếu phát hiện cổng dịch vụ 53 hoặc cấu trúc gói tin DNS, bộ giải mã tiến hành trích xuất tên miền truy vấn từ phần câu hỏi của gói tin yêu cầu, đồng thời loại bỏ các gói tin phản hồi để tối ưu hiệu năng.
 5. Nếu phát hiện dấu hiệu giao thức HTTP, bộ giải mã ứng dụng sẽ tách lọc các thông tin về phương thức yêu cầu, URI, máy chủ đích và định danh trình duyệt từ phần dữ liệu thô.
 6. Toàn bộ các thông tin trên được đóng gói đồng bộ vào một đối tượng dữ liệu duy nhất, giúp đơn giản hóa việc viết luật phát hiện ở các tầng phía sau.

3.4 Detection Engine

Module `nids/engine.py` đóng vai trò điều phối. Mỗi `PacketEvent` được đếm vào thống kê giao thức, sau đó gửi đến `SlidingWindowRules`. Các cảnh báo sinh ra được ghi xuống `AlertStore`.

Đoạn mã 3.3: Luồng xử lý một sự kiện

```
1 def process_event(self, event: PacketEvent) -> list[Alert]:
2     self.packet_count += 1
3     self.protocol_counts[event.protocol] += 1
4
5     alerts = self.rules.evaluate(event)
6     self.store.append_many(alerts)
7     self.alert_count += len(alerts)
8     return alerts
```

3.5 Luật phát hiện tích hợp

Các luật hành vi và anomaly được triển khai bằng cửa sổ trượt theo thời gian. Hệ thống lưu các sự kiện gần nhất theo IP nguồn và loại bỏ dữ liệu đã nằm ngoài khung thời gian cấu hình. Các ngưỡng trong bảng là giá trị lab/demo, chưa phải khuyến nghị production. Để lưu giữ vết thời gian của các gói tin trong cửa sổ trượt thời gian một cách tối ưu, hệ thống sử dụng hàng đợi hai đầu `collections.deque` thay vì danh sách thông thường `list`.

- **Hiệu năng chen và xóa:** Việc loại bỏ các gói tin cũ ở đầu hàng đợi đòi hỏi thao tác xóa ở đầu. Đối với Python `list`, thao tác `pop(0)` có độ phức tạp thời gian là $O(N)$ do phải dịch chuyển toàn bộ phần tử còn lại trong bộ nhớ. Đối với `deque`, thao tác `popleft()` chỉ mất thời gian $O(1)$, giúp hệ thống duy trì hiệu suất xử lý gói tin cực cao ở thời gian thực.
- **Thuật toán loại bỏ phần tử hết hạn:**

Đoạn mã 3.4: Loại bỏ sự kiện cũ trong cửa sổ thời gian

```
1 @staticmethod
2 def _drop_old_times(window: deque[float], now: float, seconds: int) -> None:
3     while window and now - window[0] > seconds:
4         window.popleft()
```

3.5.1 Cơ chế chống tấn công State Exhaustion DoS

Một trong những điểm yếu lớn nhất của các hệ thống IDS/IPS lưu vết trạng thái là nguy cơ bị tấn công State Exhaustion DoS. Kẻ tấn công có thể giả mạo hàng triệu địa chỉ IP nguồn ngẫu nhiên để vượt quá khả năng lưu trữ trạng thái của hệ thống.

nhiên để gửi gói tin qua NIDS. Nếu hệ thống tự động khởi tạo hàng đợi mới cho mỗi IP nguồn mà không có cơ chế dọn dẹp, bộ nhớ RAM của hệ thống sẽ nhanh chóng bị lấp đầy và dẫn đến sập toàn bộ NIDS.

Để giải quyết vấn đề bảo mật này, hệ thống triển khai một thuật toán dọn dẹp theo thuật toán LRU định kỳ sau mỗi 1000 gói tin được xử lý:

- Hệ thống đặt giới hạn số lượng nguồn theo dõi tối đa là 10.000 địa chỉ nguồn thông qua biến `_max_tracked_sources`.
- Khi số lượng nguồn theo dõi vượt quá giới hạn này, hàm `_cleanup_tracked_sources` sẽ tiến hành sắp xếp các địa chỉ IP theo mốc thời gian hoạt động gần nhất của chúng.
- Hệ thống sau đó tự động giải phóng một nửa số lượng địa chỉ nguồn ít hoạt động nhất để nhường chỗ cho các nguồn mới, đảm bảo dung lượng bộ nhớ RAM luôn nằm trong tầm kiểm soát an toàn.

Các luật tích hợp gồm:

Rule ID	Attack	Nhóm phát hiện	Điều kiện mặc định	Severity
RULE-001	Port Scan	Behavior-based	≥ 20 TCP destination ports từ một source trong 10 giây.	Medium
RULE-002	ICMP Ping Flood	Behavior-based	≥ 100 ICMP request packets trong 10 giây.	High
RULE-003	TCP SYN Flood	Behavior-based	≥ 100 TCP SYN packets without ACK trong 10 giây.	High
RULE-004	DNS Tunneling	Behavior/anomaly-based	≥ 5 DNS query đáng ngờ từ cùng source trong 30 giây.	Medium

Bảng 3.1: Các detection rules tích hợp trong prototype

Với port scan, prototype chỉ đếm các probe flags như SYN, FIN, NULL và Xmas. Xmas scan tương ứng FIN + PSH + URG. ACK scan có thể tạo cảnh báo giả nếu không có state tracking đầy đủ, vì vậy phần phát hiện ACK scan chỉ nên xem là minh họa trong prototype.

3.6 Luật chữ ký dạng Suricata-style subset

Ngoài luật hành vi, hệ thống hỗ trợ một tập con cú pháp kiểu Suricata/Snort. Người dùng có thể truyền tệp luật qua tham số `--rules`. Trong lần thử nghiệm được lưu, luật chữ ký được kích hoạt là luật DNS kiểm tra truy vấn chứa `example.com`:

Đoạn mã 3.5: Luật chữ ký dùng trong demo

```
1 alert dns any any -> any 53 (msg:"Custom DNS Query: example.com"; dns.query; content
  : "example.com"; nocase; classtype:policy-violation; priority:3; sid:1000101; rev
  :1;)
```

Mục tiêu của phần này là minh họa cách một signature engine hoạt động: phân tích header luật, chọn buffer cần kiểm tra, sau đó so khớp nội dung hoặc biểu thức chính quy. Đây là nguyên mẫu tập con, không phải engine tương thích đầy đủ với Suricata/Snort.

Nguyên mẫu hỗ trợ các thành phần cơ bản như phần đầu luật, nội dung, biểu thức chính quy, không phân biệt chữ hoa chữ thường, một số bộ đệm đánh DNS, nội dung gói tin và ngưỡng đơn giản. Các tính năng chưa hỗ trợ đầy đủ gồm độ sâu và độ lệch, khoảng cách và phạm vi, kiểm tra byte, cờ luồng, kiểm tra luồng TCP, dữ liệu tệp tin và tối ưu hóa mẫu quét nhanh. Tùy chọn luồng hiện chỉ được kiểm tra ở mức hướng cơ bản, chưa phải là cơ chế theo dõi luồng TCP đầy đủ trạng thái như các hệ thống thương mại.

3.7 Lưu trữ và Dashboard hiển thị

3.7.1 Lưu trữ cảnh báo dạng JSON Lines

Hệ thống sử dụng định dạng tệp JSON Lines thông qua lớp `AlertStore` để lưu trữ nhật ký cảnh báo. Mỗi cảnh báo mới xuất hiện sẽ được mã hóa thành một chuỗi JSON độc lập và kết thúc bằng ký tự xuống dòng. Quyết định thiết kế này dựa trên các đặc tính kỹ thuật quan trọng liên quan đến hiệu năng hệ thống:

- **Độ phức tạp ghi tối ưu $O(1)$:** Đối với định dạng JSON chuẩn, mỗi khi có cảnh báo mới, hệ thống bắt buộc phải đọc toàn bộ nội dung file cũ vào bộ nhớ RAM, phân tích cú pháp sang kiểu dữ liệu danh sách của Python, thêm phần tử mới, chuyển đổi lại thành chuỗi JSON và ghi đè toàn bộ tệp tin lên đĩa cứng. Thao tác này có độ phức tạp thời gian tăng dần theo độ lớn của lịch sử cảnh báo $O(M)$. Ngược lại, đối với JSON Lines, hệ thống chỉ cần mở tệp ở chế độ ghi tiếp "a" và ghi trực tiếp dòng mới vào cuối tệp tin mà không cần quan tâm đến kích thước hiện tại của tệp. Thao tác này luôn đạt độ phức tạp $O(1)$.
- **Độ tin cậy và Khôi phục:** Trong trường hợp hệ thống NIDS bị sập đột ngột, các dòng cảnh báo đã ghi trước đó vẫn hoàn toàn nguyên vẹn và không bị lỗi cấu trúc tệp tin.
- **Tương thích hệ thống lớn:** Định dạng JSON Lines là định dạng chuẩn được hỗ trợ mặc định bởi các tác nhân thu thập log nổi tiếng như Filebeat, Fluentd hoặc Logstash để đẩy dữ liệu về hệ thống SIEM tập trung.

3.7.2 Cơ chế hiển thị trên Dashboard Flask

Dashboard được xây dựng dựa trên framework Flask (`dashboard/app.py`). Để đảm bảo Dashboard luôn hiển thị thông tin trực quan mà không tạo ra gánh nặng I/O đọc tệp tin liên tục cho ổ cứng máy chủ, hệ thống áp dụng cơ chế tối ưu hóa sau:

- **Bộ nhớ đệm theo mốc thời gian sửa đổi:** Trước khi thực hiện đọc và phân tích tệp tin `alerts.jsonl`, ứng dụng web kiểm tra thuộc tính mốc thời gian sửa đổi cuối cùng của tệp tin `os.path.getmtime`. Nếu tệp tin không có thay đổi so với lần đọc trước, Dashboard sẽ trả về ngay kết quả đã được cache sẵn trong bộ nhớ RAM mà không cần thực hiện thao tác I/O đọc ổ cứng.
- **Phân tích thống kê thời gian thực:** Khi phát hiện tệp tin cảnh báo có sự thay đổi, Dashboard thực hiện đọc tệp, trích xuất dữ liệu để tính toán nhanh:
 - Tổng số lượng cảnh báo đã kích hoạt.
 - Biểu đồ thống kê số lượng cảnh báo phân nhóm theo mức độ nghiêm trọng gồm cao, trung bình, thấp.
 - Phân tích danh sách các địa chỉ IP tấn công nhiều nhất hỗ trợ khoanh vùng mối đe dọa.
 - Danh sách hiển thị chi tiết mười cảnh báo gần nhất bao gồm đầy đủ bằng chứng thu thập được từ gói tin.

Dashboard phục vụ quan sát và trình diễn kết quả; phần phản ứng tự động như chặn IP hoặc cập nhật tường lửa chưa nằm trong phạm vi bài tập.

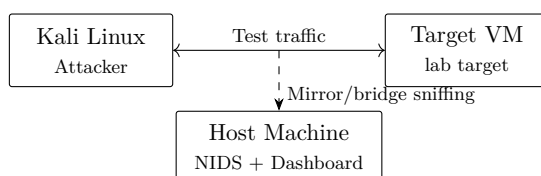
Chương 4

Kịch bản Thử nghiệm và Đánh giá

4.1 Môi trường thử nghiệm

Việc kiểm thử được thực hiện trong môi trường phòng thí nghiệm ảo hóa cô lập để tránh ảnh hưởng đến mạng thật. Các lệnh tạo lưu lượng chỉ dùng trong máy ảo có kiểm soát và không chạy trên mạng thật nếu không có quyền. Mô hình gồm ba thành phần:

- **Host Machine:** chạy NIDS engine và Flask dashboard.
- **Kali Linux VM:** đóng vai trò máy tạo lưu lượng kiểm thử bằng `nmap`, `hping3` và `dig`.
- **Target VM:** nhận lưu lượng scan, flood và DNS trong môi trường lab.



Hình 4.1: Mô hình lab kiểm thử

Trong mô hình này, NIDS sniff traffic trên interface mirror/bridge của lab. Nếu dùng SPAN/TAP hoặc bridge trong môi trường thật, cần lưu ý khả năng mất packet hoặc asymmetric routing làm giảm độ đầy đủ của dữ liệu quan sát.

4.2 Khởi động hệ thống

Trên host, dashboard được chạy ở một terminal:

Đoạn mã 4.1: Khởi động dashboard

```
1 python3 dashboard/app.py
```

NIDS engine được chạy ở terminal khác. Interface cần thay bằng interface thực tế kết nối các máy ảo:

Đoạn mã 4.2: Khởi động NIDS engine

```
1 sudo .venv/bin/python main.py --iface <YOUR_INTERFACE> --rules custom_test.rules --limit 0
```

Khi dùng PCAP thay cho live capture, có thể chạy:

Đoạn mã 4.3: Phân tích PCAP

```
1 python3 main.py --pcap sample.pcap --rules custom_test.rules
```

4.3 Các kịch bản kiểm thử

4.3.1 Quét cổng bằng nmap

Kịch bản sử dụng **nmap** để gửi SYN tới nhiều cổng đích trong thời gian ngắn:

```
1 nmap -sS -p 21,22,23,25,53,80,110,135,139,143,443,445,\
2 993,995,1433,1723,3306,3389,5900,8080,8443,8888,9090,\
3 9200,9300 --scan-delay 30ms --max-retries 1 -T3 -n --open <TARGET_IP>
```

Kết quả mong đợi là cảnh báo **RULE-001 Port Scan**, vì cùng một IP nguồn truy cập nhiều cổng TCP đích trong cửa sổ 10 giây.

4.3.2 ICMP Ping Flood

Kịch bản gửi nhiều gói ICMP Echo Request:

```
1 hping3 --icmp -c 120 -i u80000 <TARGET_IP>
```

Tùy chọn **-i u80000** tạo khoảng cách 80000 micro giây giữa các gói, giúp lưu lượng đỡ đột ngột nhưng vẫn đủ vượt ngưỡng demo ≥ 100 packets/10 giây trong lab. Kết quả mong đợi là cảnh báo **RULE-002 ICMP Ping Flood** khi bộ đếm ICMP request từ cùng nguồn vượt ngưỡng.

4.3.3 TCP SYN Flood

Kịch bản gửi nhiều gói SYN tới cổng 80:

```
1 hping3 -S -p 80 -c 120 -i u80000 <TARGET_IP>
```

Tùy chọn **-S** tạo TCP SYN without ACK, còn **-i u80000** giữ tốc độ ổn định hơn nhưng vẫn đủ cao cho threshold demo. Kết quả mong đợi là cảnh báo **RULE-003 TCP SYN Flood**. Trong phạm vi bài tập, hệ thống chỉ cảnh báo dựa trên số lượng gói SYN without ACK từ cùng nguồn, chưa kiểm tra trạng thái hoàn tất kết nối ở mức TCP stream đầy đủ.

4.3.4 DNS tunneling

Kịch bản DNS tạo các truy vấn có nhãn dài/entropy cao. Để kích hoạt cảnh báo, truy vấn cần được lặp lại ít nhất 5 lần:

```
1 for i in {1..5}; do
2   dig "aGVsbG93b3JsZGh1bGxvd2...$i.tunnel.example.com"
3 done
```

Kết quả mong đợi là **RULE-004 DNS Tunneling Suspicion** khi có đủ số truy vấn bất thường trong cửa sổ thời gian.

Rule DNS tunneling có thể false positive với một số domain hợp lệ như CDN, tracking, DKIM, ACME challenge hoặc service discovery. Trong thực tế nên kết hợp thêm whitelist, NXDOMAIN rate, số lượng unique subdomain, qtype và reputation của base domain.

4.3.5 DNS signature rule

Kịch bản DNS bình thường và DNS tunneling cũng được dùng để kiểm tra luật chữ ký tùy chỉnh. Luật trong file `custom_test.rules` so khớp các truy vấn DNS chứa chuỗi `example.com` (chỉ là domain demo, không phải độc hại):

```
1 dig +short example.com @8.8.8.8
2 dig +short <LONG_LABEL>.tunnel.example.com @8.8.8.8
```

Kết quả mong đợi là cảnh báo signature-based **1000101 Custom DNS Query: example.com**. Luật này dùng `dns.query` và `content` để minh họa cơ chế nạp rule tùy chỉnh dạng *Suricata-style subset*.

4.4 Phân tích cấu trúc nhật ký cảnh báo sinh ra (JSONL)

Dưới đây là cấu trúc chi tiết của các bản ghi cảnh báo (Alert) được NIDS tự động phát hiện và xuất ra tệp `alerts.jsonl`. Việc phân tích cấu trúc này giúp các kỹ sư an toàn thông tin hiểu rõ cơ chế thu thập chứng cứ (evidence collection) của hệ thống:

4.4.1 Cảnh báo hành vi: Quét cổng (Port Scan)

Bản ghi cảnh báo dưới đây minh họa kết quả khi phát hiện máy Kali Linux (192.168.100.15) quét cổng mục tiêu:

Đoạn mã 4.4: Bản ghi JSONL cảnh báo Port Scan

```
1 {
2   "timestamp": "2026-06-24T05:00:12.145Z",
3   "rule_id": "RULE-001",
4   "attack_type": "Port Scan",
5   "detection_method": "behavior",
6   "severity": "Medium",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "192.168.100.20",
9   "description": "192.168.100.15 contacted 20 unique TCP ports in 10 seconds",
10  "evidence": {
11    "count": 20,
12    "unique_ports": [21, 22, 23, 25, 53, 80, 110, 135, 139, 143, 443, 445, 993, 995,
13                      1433, 1723, 3306, 3389, 5900, 8080]
14  }
```

Trường `evidence` lưu giữ danh sách các cổng đã quét để phục vụ điều tra xem dịch vụ nhạy cảm nào của hệ thống có nguy cơ bị lộ lọt thông tin.

4.4.2 Cảnh báo hành vi: ICMP Ping Flood

Khi lưu lượng Ping từ một nguồn vượt quá 100 gói tin trong 10 giây, cảnh báo có dạng như sau:

Đoạn mã 4.5: Bản ghi JSONL cảnh báo ICMP Flood

```
1 {
2   "timestamp": "2026-06-24T05:01:45.321Z",
3   "rule_id": "RULE-002",
4   "attack_type": "ICMP Ping Flood",
5   "detection_method": "behavior",
6   "severity": "High",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "192.168.100.20",
```

```
9  "description": "192.168.100.15 sent 100 ICMP packets in 10 seconds",
10  "evidence": {
11    "icmp_packets": 100
12  }
13 }
```

Cảnh báo này có mức độ nghiêm trọng **High** vì nó đe dọa trực tiếp tới tính sẵn sàng của thiết bị mạng.

4.4.3 Cảnh báo hành vi: TCP SYN Flood

Cảnh báo SYN Flood ghi nhận hành vi flood cờ SYN mà không hoàn thành cái bắt tay ba bước:

Đoạn mã 4.6: Bản ghi JSONL cảnh báo TCP SYN Flood

```
1 {
2   "timestamp": "2026-06-24T05:03:10.987Z",
3   "rule_id": "RULE-003",
4   "attack_type": "TCP SYN Flood",
5   "detection_method": "behavior",
6   "severity": "High",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "192.168.100.20",
9   "description": "192.168.100.15 sent 100 SYN packets without ACK in 10 seconds",
10  "evidence": {
11    "dst_ip": "192.168.100.20",
12    "dst_port": 80,
13    "service_key": "192.168.100.15:192.168.100.20:80",
14    "syn_packets": 100,
15    "window_seconds": 10
16  }
17 }
```

Thông tin chi tiết về cổng đích (`dst_port = 80`) chỉ ra dịch vụ web đang là mục tiêu bị khai thác.

4.4.4 Cảnh báo bất thường: DNS Tunneling

Khi kẻ tấn công sử dụng công cụ tạo luồng đường truyền qua DNS, hệ thống phân tích Shannon Entropy và sinh cảnh báo:

Đoạn mã 4.7: Bản ghi JSONL cảnh báo DNS Tunneling

```
1 {
2   "timestamp": "2026-06-24T05:05:22.012Z",
3   "rule_id": "RULE-004",
4   "attack_type": "DNS Tunneling Suspicion",
5   "detection_method": "anomaly",
6   "severity": "Medium",
7   "source_ip": "192.168.100.15",
8   "destination_ip": "8.8.8.8",
9   "description": "192.168.100.15 sent 5 DNS queries with tunneling indicators in 30 seconds",
10  "evidence": {
11    "latest_query": "ahZsbG93b3JsZGh1bGxvd29ybGQ.tunnel.example.com",
12    "max_label_entropy": 4.276,
13    "max_label_length": 54,
14    "query_length": 65,
15    "reason_counts": {
16      "high_entropy_label": 5,
17      "long_label": 5,
```

```
18     "long_query_name": 5
19   },
20   "recent_queries": [
21     "aGVsbG93b3JsZGh1bGxvd2...1.tunnel.example.com",
22     "aGVsbG93b3JsZGh1bGxvd2...2.tunnel.example.com",
23     "aGVsbG93b3JsZGh1bGxvd2...3.tunnel.example.com",
24     "aGVsbG93b3JsZGh1bGxvd2...4.tunnel.example.com",
25     "aGVsbG93b3JsZGh1bGxvd2...5.tunnel.example.com"
26   ],
27   "suspicious_queries": 5
28 }
29 }
```

Trường `max_label_entropy` và `reason_counts` là bằng chứng toán học đặc lực giúp phân loại hành vi tunneling thay vì các truy vấn DNS hợp lệ bình thường.

4.5 Đánh giá kết quả

Trong môi trường lab, các kịch bản trên kiểm tra được các nhóm phát hiện chính:

- Luật hành vi phát hiện các mẫu dựa trên tần suất và số lượng sự kiện trong thời gian ngắn.
- Luật signature phát hiện chuỗi đặc trưng trong truy vấn DNS.
- Dashboard hiển thị được số lượng cảnh báo, mức độ nghiêm trọng, IP nguồn và danh sách cảnh báo gần nhất.

Số liệu dưới đây được lấy trực tiếp từ file `data/alerts.jsonl`. File này lưu 14 cảnh báo thực nghiệm trong lần chạy lab: 3 cảnh báo behavior, 1 cảnh báo anomaly và 10 cảnh báo signature DNS.

Scenario	Expected alert	Observed alert	Evidence chính	Ghi chú
Port scan	RULE-001	1 alert	unique_ports ≥ 20, count = 20	Vượt ngưỡng scan trong cửa sổ 10 giây.
ICMP flood	RULE-002	1 alert	icmp packets = 100	Trigger đúng ngưỡng demo.
SYN flood	RULE-003	1 alert	syn packets = 100, dst_port = 80	SYN without ACK.
DNS tunneling	RULE-004	1 alert	suspicious queries = 5, max label entropy = 4.276, max label length = 54	≥ 5 suspicious queries/30 giây.
DNS signature	1000101	10 alerts	dns.query chứa example.com (domain demo)	Gồm truy vấn nền và các truy vấn tunnel.example.com.

Bảng 4.1: Bảng kết quả thực nghiệm từ file alert đã lưu

Kết quả này xác nhận hệ thống đáp ứng mục tiêu học tập và trình diễn. Tuy nhiên, đánh giá này chỉ có giá trị trong phạm vi phòng thí nghiệm có kiểm soát. Để kết luận về độ chính xác trong mạng thực tế cần có bộ dữ liệu lớn hơn, lưu lượng nền đa dạng hơn, kiểm thử cảnh báo giả, cảnh báo bỏ sót và đo hiệu năng xử lý gói tin.

Chương 5

Kết luận và Hướng phát triển

5.1 Kết quả đạt được

Bài tập đã xây dựng được một hệ thống NIDS nhỏ gọn phục vụ học tập và trình diễn. Hệ thống hoàn chỉnh các thành phần chính của một pipeline phát hiện xâm nhập mạng:

- Đọc lưu lượng từ file PCAP hoặc từ live interface bằng thư viện Scapy.
- Chuẩn hóa cấu trúc gói tin thô thành đối tượng dữ liệu sự kiện chuẩn hóa `PacketEvent`.
- Phát hiện hành vi bất thường bằng cửa sổ trượt thời gian với các ngưỡng cấu hình lab/demo.
- Phân tích một Suricata-style subset để phát hiện các chữ ký trong HTTP, DNS và payload.
- Tách biệt danh sách các tên miền độc hại khỏi nhóm luật mặc định để tăng tính tùy biến.
- Ghi nhận thông tin cảnh báo ra định dạng JSON Lines và hiển thị trực quan thông qua Dashboard xây dựng bằng Flask.
- Xây dựng kịch bản demo để kiểm tra các rule chính trong môi trường lab.

Kết quả quan trọng nhất của bài tập lớn là làm rõ cách các thành phần của hệ thống phát hiện xâm nhập phối hợp với nhau. Mô hình capture cung cấp dữ liệu đầu vào, bộ parser chuẩn hóa dữ liệu, rule engine tạo cảnh báo, storage ghi nhận kết quả và dashboard hỗ trợ quan sát.

5.2 Hạn chế

Mặc dù đạt được mục tiêu học tập, hệ thống hiện tại vẫn tồn tại một số hạn chế kỹ thuật:

- **Chỉ phát hiện, chưa ngăn chặn:** Hệ thống chỉ dừng lại ở vai trò cảnh báo thụ động, chưa hỗ trợ các hành động chủ động như chặn gói tin, ngắt kết nối hoặc cập nhật cấu hình tường lửa.
- **Chưa xử lý lưu lượng mã hóa:** Các nội dung truyền tải qua giao thức HTTPS hoặc TLS không thể kiểm tra bằng chữ ký tĩnh khi chưa có cơ chế giải mã gói tin trung gian.
- **Chưa tái tạo TCP stream đầy đủ:** Payload bị chia nhỏ qua nhiều gói tin khác nhau có thể làm giảm hiệu quả của việc đối sánh signature.
- **Chưa xử lý đầy đủ fragmentation và segmentation:** Các gói tin IP fragmentation, TCP segmentation và các gói tin đến sai thứ tự có thể làm sai lệch dữ liệu đầu vào của rule engine.

- **Dễ bị evasion với payload biến đổi:** Kẻ tấn công có thể mã hóa hoặc làm rối payload để dễ dàng vượt qua các luật kiểm tra chuỗi tĩnh đơn giản.
- **Phụ thuộc vào visibility của điểm giám sát:** Cấu trúc hình cổng gương Switch hoặc thiết bị SPAN/TAP có thể bị rơi gói tin hoặc không nhìn thấy đủ hai chiều lưu lượng khi hệ thống mạng thực hiện định tuyến bất đối xứng.
- **Hiệu năng còn giới hạn:** Việc lập trình bằng ngôn ngữ Python kết hợp thư viện Scapy chỉ phù hợp cho mục đích nghiên cứu học tập, chưa thể đáp ứng môi trường mạng thực tế có băng thông lớn.
- **Phụ thuộc vào ngưỡng và luật:** Chất lượng cảnh báo phụ thuộc hoàn toàn vào việc cấu hình threshold, baseline của từng hệ thống và tính đầy đủ của tập luật chữ ký được định nghĩa.

5.3 Hướng phát triển

Các hướng cải tiến phù hợp với kiến trúc hiện tại gồm:

- Bổ sung cơ chế IP defragmentation và TCP stream reassembly để kiểm tra payload ở mức phiên truyền thông thay vì từng gói tin đơn lẻ.
- Bổ sung stateful protocol analysis cho TCP, HTTP, DNS và mở rộng parser rule tương thích sâu hơn với định dạng luật Snort hoặc Suricata-like subset.
- Tích hợp cơ chế tự học để thiết lập whitelist và baseline tự động từ lưu lượng bình thường nhằm giảm thiểu false positive.
- Xây dựng correlation engine để gom nhiều cảnh báo riêng lẻ như quét cổng, flood cờ hoặc tấn công ứng dụng web thành một sự cố an ninh duy nhất theo dòng thời gian.
- Thêm cơ chế cấu hình động các luật và ngưỡng thông qua tệp tin cấu hình YAML hoặc trực tiếp trên giao diện Dashboard.
- Thực hiện đánh giá kiểm thử trên các bộ dữ liệu lưu lượng mạng công khai lớn để báo cáo các chỉ số về precision, recall và false positive rate.
- Đo đạc hiệu năng xử lý trên các file PCAP dung lượng lớn và tối ưu hóa mã nguồn để giảm thiểu tối đa hiện tượng packet drop.
- Tích hợp phản ứng bán tự động như xuất danh sách chỉ số thỏa hiệp độc hại hoặc gửi cảnh báo qua Webhook đến tường lửa để thực hiện chặn IP nguồn tấn công trong môi trường kiểm thử.
- Xây dựng bộ kịch bản kiểm thử tự động gồm cả lưu lượng bình thường và lưu lượng tấn công để đánh giá chính xác tỷ lệ false positive và false negative.

5.4 Kết luận

Hệ thống đã đáp ứng hoàn toàn mục tiêu chính của bài tập lớn: minh họa trực quan một hệ thống NIDS hoạt động từ khâu thu thập gói tin đến sinh cảnh báo và hiển thị kết quả. Dù chỉ là một nguyên mẫu phục vụ học tập, bài tập đã cung cấp nền tảng thực hành tốt để hiểu rõ các kỹ thuật phát hiện xâm nhập mạng và các thách thức thực tế khi vận hành hệ thống giám sát an ninh mạng.

Phụ lục

Phụ lục A: Hướng dẫn cài đặt và cấu hình hệ thống

Để triển khai và chạy thử nghiệm hệ thống NIDS trong môi trường thực tế hoặc phòng thí nghiệm, người sử dụng cần thực hiện các bước cấu hình hệ điều hành và cài đặt môi trường như sau:

1. Cài đặt các thư viện phụ thuộc của hệ thống

Hệ thống yêu cầu cài đặt phiên bản Python 3.8 trở lên. Các thư viện phụ thuộc được quản lý thông qua tệp tin `requirements.txt` và có thể cài đặt bằng trình quản lý gói pip:

```
1 # Tạo môi trường ảo python
2 python3 -m venv .venv
3
4 # Kích hoạt môi trường ảo trên Linux/macOS
5 source .venv/bin/activate
6
7 # Kích hoạt môi trường ảo trên Windows
8 .venv\Scripts\activate
9
10 # Cập nhật pip và cài đặt các thư viện phụ thuộc
11 pip install --upgrade pip
12 pip install -r requirements.txt
```

2. Cấu hình quyền bắt gói tin trực tiếp trên Linux

Giao thức bắt gói tin trực tiếp trên card mạng yêu cầu tiến trình chạy dưới quyền quản trị tối cao root. Nếu chạy bằng tài khoản thông thường, thư viện Scapy sẽ báo lỗi không thể truy cập socket thô của hệ thống.

```
1 # Chạy engine NIDS với quyền sudo trên Linux
2 sudo .venv/bin/python main.py --iface eth0 --rules custom.rules
```

Để bắt gói tin ở chế độ promiscuous mode nhằm giám sát toàn bộ lưu lượng trong phân đoạn mạng chứ không chỉ các gói tin gửi trực tiếp đến máy chủ, ta cần kích hoạt tính năng này trên card mạng đích:

```
1 # Kích hoạt promiscuous mode cho card mạng eth0
2 sudo ip link set eth0 promisc on
```

3. Cấu hình hỗ trợ trên Windows

Trên hệ điều hành Windows, Scapy yêu cầu trình điều khiển Npcap hoặc WinPcap để có thể tương tác với card mạng ở tầng liên kết dữ liệu. Quy trình cài đặt bao gồm:

1. Tải xuống trình cài đặt Npcap từ trang chủ chính thức.
2. Chạy trình cài đặt với quyền Administrator.
3. Trong quá trình cài đặt, tích chọn tùy chọn tương thích với WinPcap API để đảm bảo các thư viện Python cũ hoạt động ổn định.
4. Khởi động lại máy tính hoặc khởi động lại dịch vụ card mạng để trình điều khiển Npcap nhận dạng các giao diện mạng khả dụng.

Phụ lục B: Danh sách mở rộng các luật chữ ký mẫu

Dưới đây là một số luật chữ ký mẫu được thiết kế theo cú pháp Suricata để nạp vào hệ thống nhằm mở rộng khả năng phát hiện các hành vi tấn công phổ biến:

1. Luật phát hiện tấn công ứng dụng web SQL Injection

Luật dưới đây quét phần dữ liệu yêu cầu HTTP để tìm kiếm chuỗi ký tự tấn công SQL Injection cơ bản:

```
1 alert tcp any any -> $HOME_NET $HTTP_PORTS (msg:"SQL Injection Attempt - UNION  
SELECT"; http.uri; content:"union"; nocase; content:"select"; nocase; sid  
:1000001; rev:1;)
```

Giải thích luật:

- `http.uri`: Chỉ định bộ đệm đích URI của HTTP yêu cầu để đối sánh chuỗi.
- `content:"union" ... content:"select"`: Yêu cầu cả hai chuỗi này phải xuất hiện đồng thời trong URI.
- `nocase`: Không phân biệt chữ hoa hay chữ thường đối với các chuỗi tìm kiếm.

2. Luật phát hiện tấn công chèn mã kịch bản XSS

Luật dưới đây phát hiện hành vi chèn thẻ Script độc hại qua tham số URI:

```
1 alert tcp any any -> $HOME_NET $HTTP_PORTS (msg:"XSS Attempt - Script Tag"; http.uri  
; content:"<script>"; nocase; sid:1000002; rev:1;)
```

3. Luật phát hiện truy cập tệp tin hệ thống nhạy cảm

Luật dưới đây phát hiện hành vi duyệt thư mục ngược để đọc tệp tin cấu hình mật khẩu hệ thống Linux:

```
1 alert tcp any any -> $HOME_NET $HTTP_PORTS (msg:"Directory Traversal - etc/passwd  
Access"; http.uri; content:"etc/passwd"; nocase; sid:1000003; rev:1;)
```

Phụ lục C: Bảng so sánh các giải pháp IDS nguồn mở phổ biến

Bảng so sánh dưới đây cung cấp cái nhìn tổng quan về đặc tính kỹ thuật của các giải pháp phát hiện xâm nhập mạng mã nguồn mở phổ biến nhất hiện nay trên thế giới:

Tiêu chí	Snort	Suricata	Zeek
Kiến trúc xử lý	Đơn luồng trong các phiên bản cũ, đa luồng được bổ sung ở phiên bản Snort 3.	Thiết kế đa luồng mặc định, tối ưu hóa xử lý song song trên CPU đa nhân.	Đơn luồng xử lý chính, hỗ trợ mô hình phân cụm cluster để chia tải trên nhiều luồng.
Cơ chế phát hiện	Dựa trên chữ ký và đối sánh mẫu chuỗi tĩnh là chủ yếu.	Dựa trên chữ ký tĩnh, có hỗ trợ các luật phân tích hành vi nâng cao.	Dựa trên phân tích giao thức có trạng thái và thực thi script lập trình động.
Lắp ráp luồng TCP	Có hỗ trợ thông qua các pre-processor tích hợp sẵn.	Có hỗ trợ với hiệu năng cao, xử lý tốt fragmentation và segmentation.	Hỗ trợ tái lắp ráp luồng mạnh mẽ để phân tích dữ liệu tầng ứng dụng.
Định dạng luật	Cú pháp luật dạng Snort cổ điển.	Tương thích hoàn toàn với cú pháp luật Snort, mở rộng thêm các từ khóa mới.	Sử dụng ngôn ngữ lập trình kịch bản hướng sự kiện của riêng Zeek.

Bảng 5.1: Bảng so sánh các giải pháp IDS nguồn mở phổ biến

Tài liệu tham khảo

- [1] Bộ môn Truyền thông và Mạng máy tính. *Bài giảng Nhập môn An toàn thông tin (IT4015)*.
- [2] William Stallings (2016). *Cryptography and Network Security: Principles and Practice* (7th Edition). Pearson Education.
- [3] James P. Anderson (1980). *Computer Security Threat Monitoring and Surveillance*. Technical report, James P. Anderson Co., Fort Washington, Pennsylvania.
- [4] Dorothy E. Denning (1987). *An Intrusion-Detection Model*. IEEE Transactions on Software Engineering, Vol. SE-13, No. 2, pp. 222-232.
- [5] Karen Scarfone, Peter Mell (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94.
- [6] OISF. *Suricata User Guide*. Truy cập trực tuyến tại: <https://suricata.readthedocs.io/>.
- [7] Snort Project. *Snort 3 Rule Writing Guide*. Truy cập trực tuyến tại: <https://docs.snort.org/>.
- [8] Zeek Project. *Zeek Documentation*. Truy cập trực tuyến tại: <https://docs.zeek.org/>.
- [9] Scapy Project. *Scapy Documentation*. Truy cập trực tuyến tại: <https://scapy.readthedocs.io/>.
- [10] Gordon Lyon. *Nmap Network Scanning*. Truy cập trực tuyến tại: <https://nmap.org/book/>.
- [11] The MITRE Corporation. *MITRE ATT&CK Enterprise Matrix*. Truy cập trực tuyến tại: <https://attack.mitre.org/>.